

Yuzee AI Token Lab — Developer Overview

Welcome to the repo. This document is the front door: what this product is, how the two halves fit together, how to get both running locally, and where to go next depending on whether you're touching the backend or the frontend.

Read this first, then jump to [BACKEND.md](#) or [FRONTEND.md](#) for the deep dive on your side of the codebase.

1. What this is

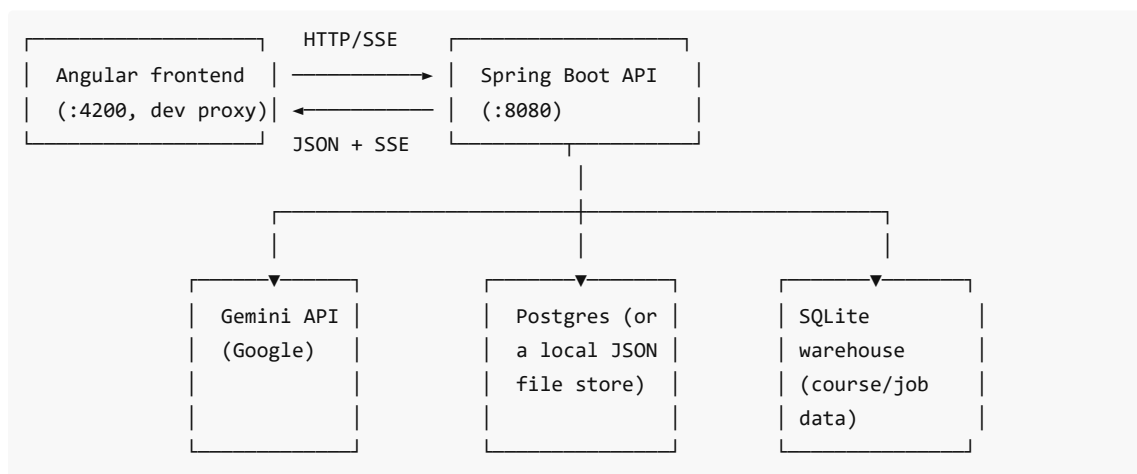
Yuzee AI Token Lab ("Yuzee" / the assistant persona "Oala") is a Gemini-powered career-counselling chat application. A user talks to an AI counsellor about career paths, skills, courses and job options; the AI replies with a structured protocol (not just prose) that the frontend renders as rich UI — question cards, comparison tables, step-by-step pathways, timelines, etc.

This repository (`yuzee-ai-lab-java`) is a **from-scratch reimplementation** of an earlier Node.js/Express + React prototype (`yuzee-ai-token-lab`), rebuilt on:

- **Backend:** Java 21 + Spring Boot 3.3.6
- **Frontend:** Angular 18 (standalone components + signals) + Bootstrap 5 + Ionic (structural shell only — see [FRONTEND.md](#))

The product behavior (the "protocol contract" the AI must follow, the counselling flows, the feature set) is intentionally the same as the original prototype. Only the implementation stack changed. Anywhere you see a comment like *"ported from server.ts"* in the Java code, that's referring to the original Node prototype — useful context, not a dependency you need to go find.

2. How the pieces fit together



- The frontend never talks to Gemini, Postgres or the warehouse directly — everything goes through the Spring Boot API.
- In dev, `ng serve` proxies any `/api/*` request to the backend on port 8080 (see `frontend/proxy.conf.json`). In production, the Angular build is copied into `backend/src/main/resources/static/` and Spring Boot serves both the API and the compiled SPA from one process/port.
- Auth is a single hardcoded admin account (no user signup/multi-tenant model) — this is a lab/demo tool, not a consumer product.

3. The core user flow

1. **Login** — one admin username/password (from env vars, defaults to `yuzeeadmin` / `yuzeeadmin@2026`). No registration flow.
2. **Start or resume a conversation** — the sidebar lists past conversations (persisted in Postgres if configured, else a local JSON file). Selecting one loads its message history; "+ New conversation" starts fresh.
3. **Chat turn** — the user types a message (or picks an option from the AI's last question). This is the heart of the app — see [BACKEND.md § How a chat turn works](#) for the full pipeline. The short version: the backend assembles context, calls Gemini, validates the response against a strict JSON protocol, optionally runs a second "fact-check" pass, and streams it all back over Server-Sent Events.
4. **The AI's reply renders as structured UI**, not a chat bubble of raw text — headings, lists, comparison tables, an active question with selectable options or a form, sometimes a whole generated pathway or a chart. The frontend's `protocol-renderer` component is what turns the JSON envelope into this UI.
5. **Side tools**, reachable from the navbar "Tools" menu, layer on top of the core chat: a mini career-pathway generator, an "Objectives" guided-activity workspace, a research/"Explore more" drill-down, a drag-and-drop pathway whiteboard, and a set of power-user panels (token usage inspector, context inspector, memory timeline, analytics, benchmark, advanced settings, export).

4. Running it locally

You need two terminals — backend and frontend — running at the same time.

Prerequisites

- Java 21+ (JDK 25 also works and is what this repo has been tested against)
- Node 18+ / npm
- A Gemini API key
- (Optional) A Postgres connection string — without one, conversations persist to a local JSON file under `backend/data/` instead. Fine for solo dev work.

Environment variables

Copy `.env.example` to a real env file (or just `source` it directly — see below) and fill in real values. **Never commit real secrets** — `.env.example` is gitignored in this repo specifically so it can hold live values for local dev without risk.

Variable	Required	Purpose
GEMINI_API_KEY	Yes	Gemini API access — nothing works without this
DATABASE_URL	No	<code>postgres://user:pass@host:port/db</code> — enables Postgres persistence; omit for the local file store
ADMIN_USERNAME / ADMIN_PASSWORD	No	Defaults to <code>yuzeeadmin</code> / <code>yuzeeadmin@2026</code>
AUTH_SECRET	No (dev) / Yes (prod)	HMAC signing secret for the admin session token. Has a hardcoded dev default — set a real value before deploying
YUZEE_WAREHOUSE_DB	No	Path to the external <code>training_gov.db</code> SQLite file (course/provider/job data, ~12GB). Without it, warehouse features report "unavailable"

WAREHOUSE_INDEX_PATH	No	Where the built FTSS search index is cached (defaults to data/warehouse/catalogue.sqlite)
CLOUDFLARE_ACCOUNT_ID / CLOUDFLARE_API_TOKEN	No	Optional Cloudflare Workers AI fallback for one routing endpoint
RESEARCH_MODEL / RESEARCH_TRUSTED_DOMAINS	No	Model + domain allowlist for the "Explore more" research feature

Spring Boot does **not** read `.env` files automatically — these need to be real shell environment variables when you start the backend.

Backend

```
cd backend
export GEMINI_API_KEY=...      # plus DATABASE_URL / ADMIN_* / etc. as needed
./mvnw spring-boot:run
```

Runs on `http://localhost:8080` . First boot takes ~15–20s.

Frontend

```
cd frontend
npm install
npm start                      # ng serve --proxy-config proxy.conf.json
```

Runs on `http://localhost:4200` , proxying `/api/*` to the backend. Log in with the admin credentials above.

Production build

`npm run build:spring` (in `frontend/`) builds the Angular app and copies it into `backend/src/main/resources/static/` , so a single Spring Boot jar/process serves both the API and the SPA on one port. There is currently **no Dockerfile or CI pipeline** in this repo — that's a real gap if a containerized deploy is planned, not an oversight to work around silently.

Tests

```
cd backend && ./mvnw test      # JUnit
cd frontend && npm test        # Karma/Jasmine
```

5. Known gaps (read before you assume something is broken)

A few subsystems are **fully built on both sides but not wired into the live request path** — this is deliberate, documented, staged work, not an accident you need to "fix" without asking first:

- ~~Warehouse retrieval not wired into a live turn~~ — **fixed:** `ChatController` now calls `WarehouseService.retrieve()` on every turn (advisory, timeout-bounded) and renders the result inline in chat via the warehouse UI components. See `BACKEND.md` § 7 and `FRONTEND.md` § 4 . Still local-file-only — pointing it at a hosted/online database remains a separate, later decision.

- **Server-side routing/microtool injection** (`RoutingPolicyService` , `BgeGateService` , `RoutingController`) exists and is testable in isolation, but `ChatController` never calls into it for a live turn — so the "route this message to the right specialist skill and inject its guidance into the prompt" behavior from the original prototype currently does nothing in production.
- A few frontend components are built but never imported: `ClarificationQuestionsModalComponent` , `ObjectiveSuggestionsComponent` . One is entirely unbuilt: `SkillSuggestionsComponent` (empty directory).
- `GET /api/tokens/utility-stats` always returns zeroes — the counter it should read from was never wired up.

None of these are silent bugs — they're documented stopping points from an in-progress migration. If you pick up one of these, check with the team on priority/scope before wiring it in, since each touches a live request path.

6. Where to go next

- Working on Java, Spring, Gemini integration, persistence, the protocol validator, or the warehouse pipeline?
→ [BACKEND.md](#)
- Working on Angular components, the chat UI, protocol rendering, or the Ionic/ Bootstrap shell? →
[FRONTEND.md](#)